OM 522 · OPTION 3 · PROTOTYPE

Planning delivery routes

Which truck serves which customers, and in what order

Presenters: A · B · C



A real plan from our tool: 45 customers, 8 trucks

The decision we automate

Presenter A

Every morning a dispatcher turns four data files into the day's delivery routes.

1

Serve every customer once

Service must start inside the customer's time window. Arriving early means waiting.

2

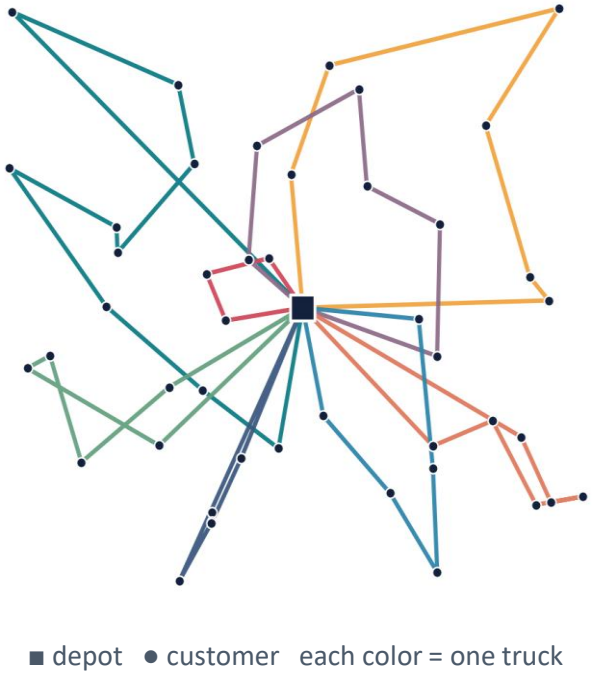
Respect every truck

Load capacity and shift end; each route starts and ends at the depot.

3

Minimize total travel minutes

We also report trucks used and computation time.



Reject bad files before planning

Presenter A

```
73  raw = pl.read_csv(file, infer_schema=False)
    → Read the file, but keep every value as plain text for now.
77  if column not in raw.columns:
    → Is a column we need missing?
78      raise InputError(f"{name}: required column '{column}' is missing")
    → Then stop, and say exactly which file and column.
```

src/vrptw/instance.py, 3 of its lines. The same file also checks types, IDs, and service times.

? What was unclear in the AI draft

The draft let Polars guess column types, so a bad value crashed without naming the file or column.

✓ How we checked it

Deleted window_end in a copy: the error names file and column, exit code 2, no plan written.

ACTUAL OUTPUT

```
Input error: customers.csv: required column 'window_end' is missing
```

The rule: go to the nearest stop that still fits

Presenter B

```
25  if load + demand[c] > inst.cap[v]:
26      continue
    → Customer c won't fit in the truck? Skip them.
33  key = (T[current][c], ids[c])
    → Rank the rest by travel time from here; a tie goes to the lower customer ID.
```

src/vrptw/rule.py, 3 of its lines. Lines 27–32 skip a customer the same way if the window or shift end would be missed.

? What was unclear in the AI draft

Does “nearest” include waiting? The brief says travel time only, ties broken by customer ID.

✓ How we checked it

Matches the instructor's worked example exactly: same visits, 352.417 travel minutes, 2 trucks.

ACTUAL OUTPUT

```
WORKED EXAMPLE: rule travel=352.417; independent checker passed
```

Researched method: tear down and rebuild

Presenter B

1

Build

Start from the rule's plan and an insertion plan; keep the better one.

›

2

Improve

Small moves that cut travel and keep every window.

›

3

Rebuild

Remove a few customers; reinsert each where it fits best.

›

4

Keep the best

Sometimes accept a worse draft to escape a dead end; never lose the best plan.

↻ **Steps 3 and 4 repeat 8,000 times**

522

fixed random seed

8,000

iterations = stopping rule

480 s

safety cap only

Source: Ropke & Pisinger (2006), Transportation Science 40(4), 455–472. A heuristic: it does not guarantee the best possible plan.

Check every plan independently

Presenter C

```
62  arrival = clock + leg
    → Arrival time = the time we left + the drive.
63  start = max(arrival, float(customer["window_start"]))
    → Arrived early? Wait until the customer's window opens.
65  if start > float(customer["window_end"]) + TOL:
    → Started after the window closed? Record a violation.
```

src/vrptw/checker.py, 3 of its lines. It uses no code from either solver.

? What was unclear in the AI draft

Trust the solver's reported arrival times? No: the checker recomputes every time from the visits.

✓ How we checked it

Broke a plan on purpose (reversed a route, dropped C006): both errors caught. 37 tests pass.

ACTUAL OUTPUT

```
V01: service at C002 starts at 381.686000, after its window end 204
customers not served: ['C006']
```

Live: the real tool on a fresh copy

Presenter A → C

```
$ pixi run prototype-demo
FRESH INPUT: out/prototype_demo/run_…/fresh_medium
RULE   1683.397 min · 11 trucks
  INDEPENDENT CHECK: feasible true · 45 served
LNS    1040.678 min · 8 trucks
  INDEPENDENT CHECK: feasible true · 45 served

$ pixi run prototype-demo --checks-only
bad file rejected · edge case · worked example
broken plan caught · 37 passed
```

WHAT TO POINT AT

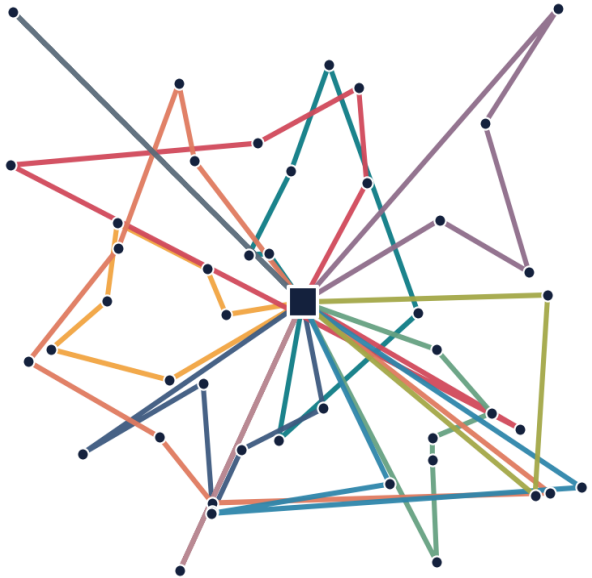
1. **The new input path**
   a fresh copy, not the original files
2. **feasible: true, twice**
   the checker passes both plans
3. **The output folder**
   a real decision.json and summary.json

Same 45 customers, two plans

Presenter B

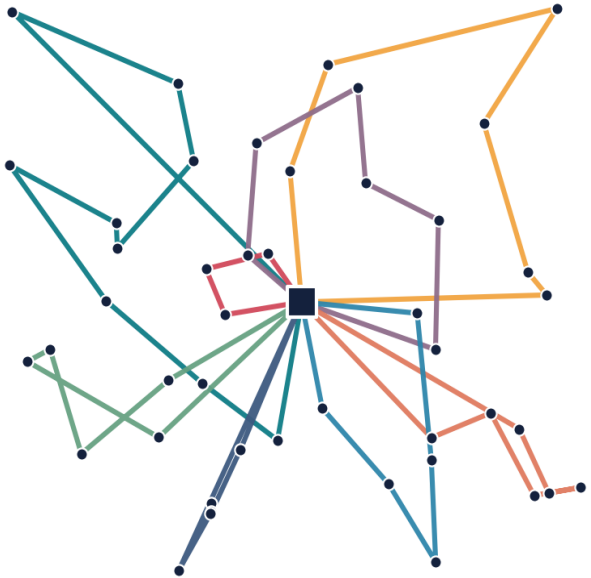
Rule to beat

1,683.4 min · 11 trucks



LNS (researched)

1,040.7 min · 8 trucks



−38%

travel minutes
(642.7 fewer)

−3

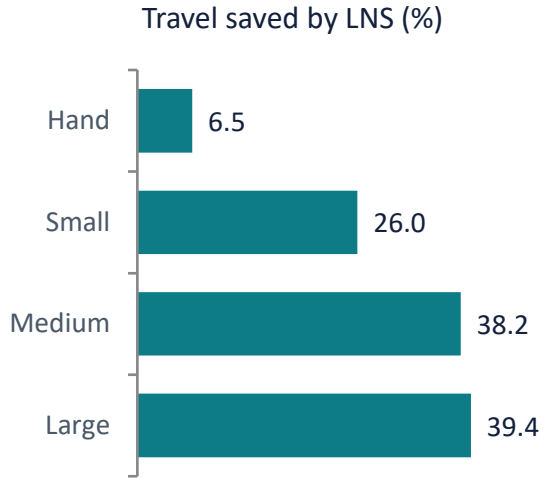
trucks
(11 → 8)

Less travel on all four supplied cases

Presenter B

Case	Travel: rule → LNS (min)	Trucks	LNS time
Hand (6)	352.4 → 329.7	2 → 2	0.5 s
Small (15)	620.6 → 459.1	5 → 4	1.0 s
Medium (45)	**1,683.4 → 1,040.7**	**11 → 8**	**8.9 s**
Large (120)	3,523.2 → 2,135.6	25 → 19	27.8 s

The rule takes under 0.003 s on every case. All 8 plans pass the independent checker. LNS: seed 522, 8,000 iterations, 480 s safety cap. Timings from one Windows 11 laptop (Intel 12th gen, Python 3.12, single thread).



Savings grow from 6.5% on 6 customers to about 39% on 120. These are four synthetic cases, not a promise for every network.

Next: what if traffic is slower?

Presenter C

Uncertain input for the final: **travel times**

1

Generate 30 traffic days

from a fixed seed, saved to files before anything runs

2

Run both plans unchanged

the same 30 scenarios for the rule and the LNS

3

Count what breaks

late service starts and late returns, plus travel and trucks

Limitation we will test: shorter routes leave less slack, so a plan that is on time with today's travel table can run late in traffic.